UNIT II

Functions, Strings & Lists

Built-in functions & modules · Defining functions · Scope · Default, keyword, *args, **kwargs · Command-line arguments · Strings · Lists · Sample Programs 7–12

➤ What you will learn in this unit

- Writing and calling your own **functions**, passing arguments in different ways, and how variable **scope** works.
- Working with **strings** — indexing, slicing, methods and formatting.
- Working with **lists** — operations, slicing, built-in functions, methods and **del**.
- Solutions of the sample programs **7 to 12** with real output.

PART A — FUNCTIONS

1. Built-in functions and commonly used modules

A **function** is a named block of code that performs a task and can be reused. Python already provides many **built-in functions** — no import needed.

Function	Purpose	Example → result
`print() input()`	Output / input	
`len(x)`	Number of items	`len("abc")` → 3
`type(x) id(x)`	Type / memory identity	`type(5)` → int
`max() min() sum()`	Largest / smallest / total	`sum([1,2,3])` → 6
`abs() round() pow()`	Absolute value / rounding / power	`round(3.678, 2)` → 3.68
`sorted() reversed()`	Sort / reverse a sequence	`sorted([3,1,2])` → [1,2,3]
`range() enumerate() zip()`	Sequences and pairing	`list(range(3))` → [0,1,2]
`int() float() str() list()`	Type conversion	`int("7")` → 7

A **module** is a file of ready-made functions. Use **import** to bring it in: `import math`, `from math import sqrt`, `import random as r`.

Module	Commonly used functions
math	`sqrt() pow() ceil() floor() factorial() pi sin() log()`
random	`random() randint(a,b) choice(seq) shuffle(list)`
datetime	`datetime.now() date.today()`
sys	`sys.argv sys.exit() sys.version`
os	`os.getcwd() os.listdir() os.path`

```
import math, random
print(math.sqrt(49), math.ceil(4.2), math.floor(4.8), math.factorial(5))
```

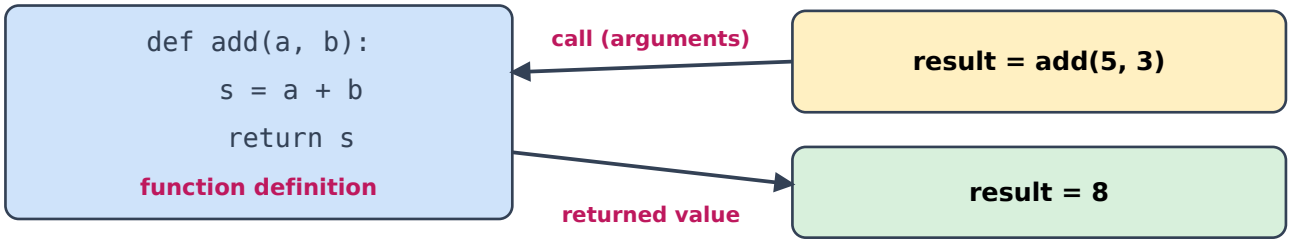
```
print(round(math.pi, 4))
random.seed(1)                          # fixed seed so the demo repeats
print(random.randint(1, 10) in range(1, 11))
print(abs(-7), pow(2, 5), max(4, 9, 2), sum([1, 2, 3]))
```

Output

```
7.0 5 4 120
3.1416
True
7 32 9 6
```

2. Function definition and calling; return statement; void function

Syntax: `def function_name(parameters):` followed by an **indented body**. The function runs only when it is **called**.



Term	Meaning
Parameter	Variable in the function definition (*a, b* above)
Argument	Actual value passed when calling (*5, 3*)
return statement	Sends a value back to the caller and **ends** the function
Void function	A function with no return statement (or plain *return*) — it gives back **None**; used for actions such as printing

```
def add(a, b):              # function with return value
    return a + b

def greet(name):            # void function (no return)
    print("Hello,", name)

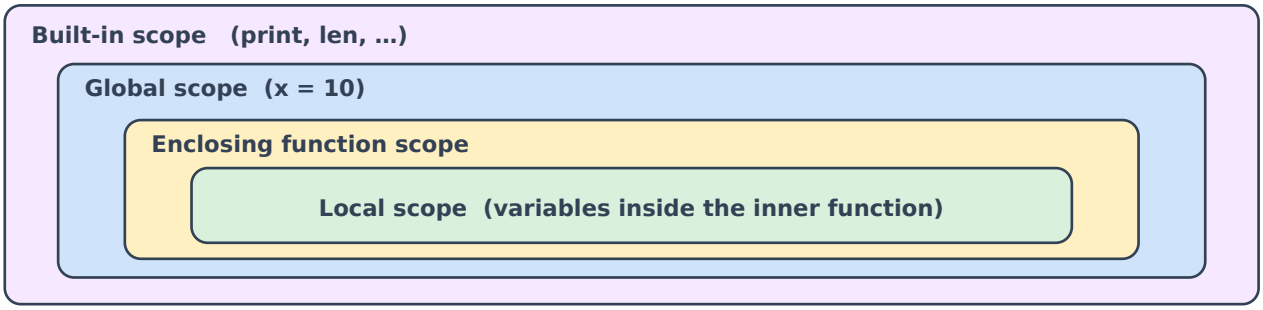
print(add(5, 3))
r = greet("Asha")
print(r)                    # void function returns None
```

Output

```
8
Hello, Asha
None
```

3. Scope and lifetime of variables

- **Scope** = the part of the program where a variable can be used. **Lifetime** = how long it stays in memory.
- **Local** variable — created inside a function; exists only **while the function runs**.
- **Global** variable — created outside all functions; lives until the program ends.
- To **change** a global variable inside a function, declare it with the **global** keyword.
- Python looks up names in the order **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in (LEGB rule).




```
x = 10                      # global

def show():
    y = 5                   # local
    print("inside :", x, y)

def change():
    global x                # modify the global variable
    x = 99

show()
change()
print("outside:", x)
# print(y)                  # NameError: y exists only inside show()
```

Output

```
inside : 10 5
outside: 99
```

4. Ways of passing arguments

Type	Description	Example call
Positional	Matched by **position** (order matters)	`power(2, 3)`
Default parameters	Parameter has a default value used when argument is missing	`def power(a, b=2)`
Keyword arguments	Pass by **name** — order does not matter	`power(b=3, a=2)`
***args**	Collects any number of extra positional arguments into a **tuple**	`def f(*args)`
****kwargs**	Collects extra keyword arguments into a **dictionary**	`def f(**kwargs)`

```
def power(a, b=2):                  # b has a default value
    return a ** b
print(power(5), power(2, 3), power(b=4, a=2))   # default / positional / keyword

def total(*args):                   # any number of positional arguments
    return sum(args)
```

```
print(total(1, 2, 3, 4))

def info(**kwargs):                    # any number of keyword arguments
    for k, v in kwargs.items():
        print(k, "=", v)
info(name="Asha", age=18)
```

Output

```
25 8 16
10
name = Asha
age = 18
```

! Careful!

- Parameters with default values must come **after** those without defaults: *def f(a, b=2)* is correct, *def f(a=2, b)* is an error.
- Order in a definition: normal → default → *args → **kwargs.

5. Command-line arguments

Values typed after the program name when running it from the terminal are **command-line arguments**. They are available in the list **sys.argv** (*argv[0]* is the script name; every value is a **string**).

```
# file: add.py
import sys
print("Script name:", sys.argv[0])
a = int(sys.argv[1])
b = int(sys.argv[2])
print("Sum =", a + b)
```

Run it as: `python add.py 10 20`

Output

```
Script name: add.py
Sum = 30
```

i Remember

- *len(sys.argv)* gives the number of arguments (including the script name).

PART B — STRINGS

6. Creating and storing strings

A **string** is a sequence of characters written in single, double or triple quotes. Strings are **immutable** — once created they cannot be changed (methods return a *new* string).

```
s1 = 'Python'
s2 = "Programming"
s3 = """This is a
multi-line string"""
print(s1, s2)
print(s3)
print("It's fine", 'He said "hi"', "Tab:\tNewline:\\n")   # quotes and escape sequences
```

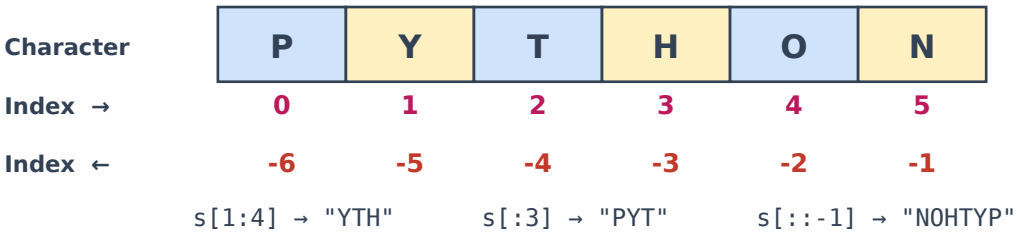
Output

```
Python Programming
This is a
multi-line string
It's fine He said "hi" Tab:	Newline:\n
```

Basic string operations

Operation	Example	Result
Concatenation +	`"Hello" + " World"`	Hello World
Repetition *	`"ab" * 3`	ababab
Length len()	`len("Python")`	6
Membership in	`"th" in "Python"`	True
Comparison	`"apple" < "banana"`	True (dictionary order)

7. Accessing characters, slicing and joining



- **Indexing**: *s[0]* is the first character, *s[-1]* the last. Index out of range → *IndexError*.
- **Slicing**: *s[start:stop:step]* — start included, **stop excluded**. Missing start = 0, missing stop = end, *step* = *-1* reverses.
- **Joining**: "*sep*".*join(list_of_strings)* combines items using the separator; *split()* does the opposite.

```
s = "PYTHON"
print(s[0], s[-1], s[2])
print(s[1:4], s[:3], s[3:], s[::2], s[::-1])
words = ["Learn", "Python", "Fast"]
joined = " ".join(words)
print(joined)
print(joined.split(" "))
```

Output

```
P N T
YTH PYT HON PTO NOHTYP
Learn Python Fast
['Learn', 'Python', 'Fast']
```

8. String methods and formatting

Method	Purpose	Example → result
`upper() lower() title()`	Change case	`"hi".upper()` → HI
`capitalize() swapcase()`	First capital / swap case	`"python".capitalize()` → Python
`strip() lstrip() rstrip()`	Remove spaces at ends	`" a ".strip()` → a
`find(sub) index(sub)`	Position of substring (find gives -1 if absent)	`"hello".find("l")` → 2
`count(sub)`	Number of occurrences	`"banana".count("a")` → 3
`replace(old, new)`	Replace text	`"cat".replace("c","b")` → bat
`startswith() endswith()`	Check beginning / end	`"abc".startswith("a")` → True
`isdigit() isalpha() isalnum()`	Check kind of characters	`"123".isdigit()` → True
`split() join()`	String ↔ list	`"a b".split()` → ['a','b']

Formatting strings

```
name, marks = "Asha", 92.456
print("Name: %s, Marks: %.1f" % (name, marks))        # % formatting
print("Name: {}, Marks: {:.2f}".format(name, marks))  # str.format()
print(f"Name: {name}, Marks: {marks:.2f}")            # f-string (best)
print(f"{name:>8}|{name:<8}|{name:^8}|")              # alignment
print(f"{255:b} {255:x} {1234567:,}")                 # binary, hex, thousands
```

Output

```
Name: Asha, Marks: 92.5
Name: Asha, Marks: 92.46
Name: Asha, Marks: 92.46
    Asha|Asha    |  Asha  |
11111111 ff 1,234,567
```

PART C — LISTS

9. Lists: creating and basic operations

A **list** is an ordered, **mutable** collection written in square brackets. It can hold items of different types and allows duplicates.

```
nums = [10, 20, 30, 40, 50]
mixed = [1, "two", 3.0, [4, 5]]          # lists can hold any type, even lists
print(nums, len(mixed))
print(nums + [60], [0] * 3)              # concatenation and repetition
print(20 in nums, 99 in nums)            # membership
nums[1] = 25                             # lists are mutable
print(nums)
for item in nums:                        # iteration
```

```
    print(item, end=" ")
print()
```

Output

```
[10, 20, 30, 40, 50] 4
[10, 20, 30, 40, 50, 60] [0, 0, 0]
True False
[10, 25, 30, 40, 50]
10 25 30 40 50
```

Indexing and slicing in lists

Works exactly like strings: *L[i]*, negative indexes, and *L[start:stop:step]*. A slice returns a **new list**.

```
L = [10, 20, 30, 40, 50, 60]
print(L[0], L[-1], L[1:4], L[:2], L[3:], L[::2], L[::-1])
L[1:3] = [21, 31]                          # slice assignment
print(L)
```

Output

```
10 60 [20, 30, 40] [10, 20] [40, 50, 60] [10, 30, 50] [60, 50, 40, 30, 20, 10]
[10, 21, 31, 40, 50, 60]
```

Built-in functions used on lists

Function	Result
`len(L) max(L) min(L) sum(L)`	Size, largest, smallest, total of items
`sorted(L)`	A **new** sorted list (original unchanged)
`list(x)`	Converts a string / tuple / range into a list
`any(L) all(L)`	True if any / all items are True
`enumerate(L) reversed(L)`	Index–value pairs / reverse iterator

List methods

Method	Purpose
`append(x)`	Add x at the **end**
`insert(i, x)`	Insert x at index i
`extend(iterable)`	Add all items of another list
`remove(x)`	Delete the **first** occurrence of value x
`pop(i)`	Remove and **return** item at index i (last if i omitted)
`index(x) count(x)`	Position of x / number of occurrences
`sort() reverse()`	Sort / reverse the list **in place**
`copy() clear()`	Shallow copy / remove everything

```
L = [30, 10, 20]
L.append(40); L.insert(1, 15); L.extend([5, 5])
print(L)
L.remove(5); print(L.pop(), L)
L.sort(); print(L)
L.reverse(); print(L, L.index(20), L.count(10))
```

Output

```
[30, 15, 10, 20, 40, 5, 5]
5 [30, 15, 10, 20, 40]
[10, 15, 20, 30, 40]
[40, 30, 20, 15, 10] 2 1
```

The del statement

del deletes an item, a slice, or the whole variable. (*remove()* deletes by value, *pop()* by index and returns it, *del* by index/slice and returns nothing.)

```
L = [10, 20, 30, 40, 50, 60]
del L[0]                  # delete by index
print(L)
del L[1:3]                # delete a slice
print(L)
del L                     # delete the whole list
# print(L)               # NameError: name 'L' is not defined
```

Output

```
[20, 30, 40, 50, 60]
[20, 50, 60]
```

★ Study Tip

- Strings are **immutable**, lists are **mutable** — a favourite exam question.
- **sort()** changes the list and returns None; **sorted()** returns a new list.

✓ Quick Revision

- **def name(params):** … **return**. No return → None (void function). Local vs global scope (global keyword), LEGB.
- Arguments: positional, default, keyword, ***args** (tuple), ****kwargs** (dict). Command-line arguments → **sys.argv**.
- Strings: immutable; index s[i], slice s[a:b:c]; methods upper, lower, strip, find, count, replace, split, join; f-strings for formatting.
- Lists: mutable; append, insert, extend, remove, pop, sort, reverse, index, count; del removes by index/slice.
- Built-ins on lists: len, max, min, sum, sorted.

SAMPLE PROGRAMS 7 – 12 (with output)

Program 7

Question

- Write a program to define a function with multiple return values.

Program

```
def calculate(a, b):
    add = a + b
    sub = a - b
    mul = a * b
    div = a / b
    return add, sub, mul, div        # returns a tuple of four values

result = calculate(20, 4)
print("Returned tuple:", result)

s, d, m, q = calculate(20, 4)        # unpack the values
print("Sum        =", s)
print("Difference =", d)
print("Product    =", m)
print("Quotient   =", q)
```

Output

```
Returned tuple: (24, 16, 80, 5.0)
Sum        = 24
Difference = 16
Product    = 80
Quotient   = 5.0
```

i Remember

- Python returns multiple values as a **tuple**; they can be unpacked into separate variables.

Program 8

Question

- Write a program to define a function using default arguments.

Program

```
def student_info(name, course="B.Tech", year=1):     # default arguments
    print(f"Name: {name}, Course: {course}, Year: {year}")

student_info("Asha")                          # uses both defaults
student_info("Ravi", "AI & ML")               # overrides course
student_info("Kiran", year=2)                 # keyword argument
student_info(year=3, name="Divya", course="CSE")

def interest(principal, rate=5, time=2):
    return principal * rate * time / 100
print("Interest with defaults:", interest(10000))
print("Interest with rate=8  :", interest(10000, 8))
```

Output

```
Name: Asha, Course: B.Tech, Year: 1
Name: Ravi, Course: AI & ML, Year: 1
Name: Kiran, Course: B.Tech, Year: 2
Name: Divya, Course: CSE, Year: 3
Interest with defaults: 1000.0
Interest with rate=8  : 1600.0
```

Program 9

Question

- Write a program to find the length of the string without using any library functions.

Program

```
s = "Python Programming"

count = 0
for ch in s:            # visit every character and count it
    count += 1

print("String :", s)
print("Length :", count)
```

Output

```
String : Python Programming
Length : 18
```

i Remember

- Here *len()* is not used — each character is counted with a loop.

Program 10

Question

- Write a program to check if the substring is present in a given string or not.

Program

```
text = "Python is easy to learn"
sub = "easy"

# Method 1: membership operator
if sub in text:
    print(f"'{sub}' is present in the string")
else:
    print(f"'{sub}' is not present")

# Method 2: find() returns the position or -1
pos = text.find("hard")
print("find('hard') =", pos, "-> not present" if pos == -1 else "-> present")

# Method 3: manual check without library methods
sub2 = "learn"
found = False
for i in range(len(text) - len(sub2) + 1):
    if text[i:i + len(sub2)] == sub2:
        found = True
        break
```

```
print(f"'{sub2}' present (manual check):", found)
```

Output

```
'easy' is present in the string
find('hard') = -1 -> not present
'learn' present (manual check): True
```

Program 11

Question

- Write a program to perform the given operations on a list: i. Addition ii. Insertion iii. slicing

Program

```
L = [10, 20, 30, 40, 50]
print("Original list:", L)

# i. Addition (adding elements)
L.append(60)                    # add one item at the end
L.extend([70, 80])              # add many items
print("After addition  :", L)
print("List + list     :", L + [90, 100])

# ii. Insertion (at a given position)
L.insert(2, 25)                 # insert 25 at index 2
print("After insertion :", L)

# iii. Slicing
print("L[2:5]  :", L[2:5])
print("L[:3]   :", L[:3])
print("L[4:]   :", L[4:])
print("L[::2]  :", L[::2])
print("L[::-1] :", L[::-1])
```

Output

```
Original list: [10, 20, 30, 40, 50]
After addition  : [10, 20, 30, 40, 50, 60, 70, 80]
List + list     : [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
After insertion : [10, 20, 25, 30, 40, 50, 60, 70, 80]
L[2:5]  : [25, 30, 40]
L[:3]   : [10, 20, 25]
L[4:]   : [40, 50, 60, 70, 80]
L[::2]  : [10, 25, 40, 60, 80]
L[::-1] : [80, 70, 60, 50, 40, 30, 25, 20, 10]
```

Program 12

Question

- Write a program to perform any 5 built-in functions by taking any list.

Program

```
nums = [45, 12, 78, 33, 5, 91]

print("List        :", nums)
print("len()       :", len(nums))              # number of items
print("max()       :", max(nums))              # largest
print("min()       :", min(nums))              # smallest
print("sum()       :", sum(nums))              # total
print("sorted()    :", sorted(nums))           # ascending (new list)
print("sorted(rev) :", sorted(nums, reverse=True))
print("Average     :", round(sum(nums) / len(nums), 2))
```

Output

```
List        : [45, 12, 78, 33, 5, 91]
len()       : 6
max()       : 91
min()       : 5
sum()       : 264
sorted()    : [5, 12, 33, 45, 78, 91]
sorted(rev) : [91, 78, 45, 33, 12, 5]
Average     : 44.0
```

Possible Exam Questions

Short answer (2–3 marks)	Long answer (5–10 marks)
1. What is a function? Give its syntax. 2. Differentiate parameter and argument. 3. What is a void function? 4. What are *args and **kwargs? 5. Differentiate local and global variables. 6. What is string slicing? Give an example. 7. Differentiate sort() and sorted(). 8. Differentiate remove(), pop() and del.	1. Explain the types of function arguments with examples. 2. Explain scope and lifetime of variables with the LEGB rule. 3. Explain command-line arguments with a program. 4. Explain any ten string methods with examples. 5. Explain the different ways of formatting strings. 6. Explain list methods and built-in functions with examples. 7. Write programs 7–12 from the sample experiments.